

Unified Education Interface — Prisma ORM Layer

Go 2 (Week 5): Migrations & Repository Layer

Repository	uei-backend
Commit	cae1f29 — “Go 2 (Week 5): baseline migration + repository/query layer”
Author	Kashish Soni (finalyearproject.uei.27@gmail.com)
Builds on	Go 1 (Week 1) — project setup + schema.prisma, all 14 tables / 8 enums
Date	September 24, 2026

1. What This Submission Covers

Go 1 delivered a hand-written `schema.prisma` matching the live, already-running `schema_v2.sql` database (14 tables, 8 enums, every foreign key / unique constraint / index) and a connection smoke test. Go 2 turns that schema into something the rest of the application can actually run on top of: it fixes a bug found in the Go 1 schema, records the existing database as a formal Prisma migration history (rather than leaving Prisma unaware of how the schema came to exist), and adds a first service/repository layer with real, runnable queries against four of the product's core workflows.

- **Bug fix carried over from Go 1** — all 8 enums now map to their real Postgres type names.
- **`prisma/migrations/0_init`** — a baseline migration generated from the live database and replayed against an empty database to prove it is correct.
- **`src/repositories/`** — four repository modules (student, risk, faculty, analytics) covering 12 query functions used across the Student, Faculty and HOD-facing dashboards.
- **`src/demo-queries.ts`** — a single runnable script that exercises the whole layer against the seeded database and prints the results.

2. Bug Fix Carried Over From Go 1

Problem. None of the 8 enums in the Go 1 `schema.prisma` declared an `@@map(...)`. Left as-is, running a migration from that schema would have told Postgres to create brand-new types named `UserRole`, `CourseType`, etc., instead of reusing the real, already-existing `snake_case` types (`user_role`, `course_type`, ...) that the live database actually has. That would have broken the baselining step in Section 3 outright, since Prisma's generated SQL would not match what is really in the database.

Fix. Added `@@map("snake_case_name")` to all 8 enums:

```
enum UserRole {
  STUDENT
  FACULTY
  HOD
  ...

  @@map("user_role")
}
```

The 8 enums fixed: `user_role`, `course_type`, `attendance_status`, `internship_status`, `achievement_category`, `risk_level`, `skill_proficiency`, `placement_outcome`. Re-verified

against the live database after the fix — no other mismatches found.

3. Baseline Migration — `prisma/migrations/0_init`

Why baselining, not a normal migration history. `schema_v2.sql` was written and applied to the database before Prisma entered the picture. Prisma Migrate normally builds up `prisma/migrations/` incrementally through `migrate dev`, but here there is nothing to “diff” against — the job is to tell Prisma “this schema already exists, treat it as already applied,” which is exactly what Prisma calls baselining.

How `0_init/migration.sql` was produced (not hand-typed):

- Ran `pg_dump --schema-only --no-owner --no-privileges --no-comments` against the live, seeded, already-verified UEI database — the same database schema `.prisma` was checked against in Go 1.
- `scripts/generate_baseline_migration.py` reformats that raw dump into Prisma Migrate's own section conventions: grouped `-- CreateEnum / -- CreateTable / -- CreateIndex / -- AddForeignKey` blocks, type names normalized (`character varying(n)` → `VARCHAR(n)`, etc.), unique constraints emitted as `CREATE UNIQUE INDEX` the way Prisma itself does.
- **Verification:** the generated `migration.sql` was replayed against a brand-new, empty database and the resulting schema was confirmed identical to the source — before being committed.

Baselining commands (run once, locally, against your own DB):

```
npx prisma migrate resolve --applied 0_init
npx prisma migrate status # → "Database schema is up to date!"
```

After that, any further schema change goes through the normal `npx prisma migrate dev --name <change> flow`.

4. Repository / Service Layer — src/repositories/

Four modules, 12 exported functions total, each wired to a real Prisma Client query rather than a stub. Grouped by the dashboard/workflow each one serves.

4.1 student.repository.ts

Function	What it does
getStudentDigitalTwin(studentId)	The single “Digital Twin” call the Student/HOD dashboards need: one student's profile, account, course enrollments + course details, skills, internships, achievements, semester history, placement readiness and open risk flags — all via one Prisma <code>include</code> . Replaces 6–7 hand-written JOINS with one round trip.
listStudentsBySection(section)	Lightweight roster query for a section — roll number, name, semester, CGPA, attendance — sorted by roll number.
createStudentWithAccount(input)	Creates a User + StudentProfile as one atomic <code>\$transaction</code> : if the profile insert fails (e.g. duplicate roll number), the user row is rolled back too, so there is never an orphaned login with no profile.

4.2 risk.repository.ts

Function	What it does
listWeakStudents(level?)	The weak-student radar list for faculty/HOD dashboards, optionally filtered by <code>RiskLevel</code> , newest flags first, with student + course context inlined.
getStudentRiskHistory(studentId)	Full risk-flag history for one student across every course they were flagged in.
markIntervention(radarId, facultyId, notes)	Records a faculty intervention against a flagged risk entry (notes, reviewer, timestamp, marks it intervened).
getRiskLevelBreakdown()	Count of open flags per risk level via <code>groupBy</code> — feeds a dashboard stat tile.

4.3 faculty.repository.ts

Function	What it does
getFacultyCourseLoad(facultyId)	A faculty member's “My Courses” page: every section they teach, ordered by academic year then semester.
markAttendance(enrollmentId, facultyId, date, status)	Marks one attendance record via <code>upsert</code> keyed on the <code>@@unique([enrollmentId, attendanceDate])</code> constraint — re-marking the same session updates the status instead of throwing a unique-constraint error, so it's safe to re-run.
getEnrollmentAttendancePct(enrollmentId)	Attendance percentage for one enrollment, computed from its recorded sessions (present / total, rounded to one decimal).

4.4 analytics.repository.ts

Function	What it does
getPlacementStats()	Placement outcome breakdown — count, average package, average readiness score per <code>placementStatus</code> , via <code>groupBy + _avg</code> .
getSectionPerformance()	Average CGPA and attendance per section, plus student count — a quick HOD-level comparison across sections.

Function	What it does
<code>getCourseBacklogRates()</code>	Per-course pass/backlog breakdown: enrolled count, backlog count and backlog rate %, computed from current enrollments.

5. Demo Runner — `src/demo-queries.ts`

A single script that exercises the repository layer end-to-end against the seeded database and prints results with `console.table`: a section roster, one student's full Digital Twin, the risk-level breakdown, the top 5 HIGH-risk flags, placement stats, section performance, and the top 5 courses by backlog rate. This is the fastest way to demo the whole Go 2 layer in one command.

```
npm run demo:queries
```

6. How to Run This Submission

Assumes Go 1's setup (dependencies installed, `.env` pointed at a database that already has `schema_v2.sql` applied):

```
# 1. Bring schema.prisma's enum fix in and regenerate the client
npx prisma validate
npx prisma generate

# 2. Baseline the existing schema into Prisma's migration history
npx prisma migrate resolve --applied 0_init
npx prisma migrate status      # → "Database schema is up to date!"

# 3. Run the repository layer against your seeded data
npm run demo:queries
```

`package.json` script shortcuts added this submission: `migrate:resolve-baseline` (`prisma migrate resolve --applied 0_init`), `migrate:status`, and `demo:queries` (`ts-node src/demo-queries.ts`).

7. Verification Performed

- **Enum fix:** re-checked all 8 enums against the live database's actual Postgres type names after adding `@@map`.
- **Baseline migration:** `0_init/migration.sql` was generated from a real `pg_dump` of the verified database (not hand-typed), then replayed against a brand-new empty database and confirmed to reproduce an identical schema before being committed.
- **Repository layer:** all 12 functions are real Prisma Client calls (no stubs/mocks) and are exercised together by `demo-queries.ts` against the seeded database.

8. Files Changed This Submission

File	Diff	Purpose
<code>README.md</code>	+44 / -5	Go 2 section documenting this submission
<code>package.json</code>	+5 / -0	migrate/demo npm script shortcuts

File	Diff	Purpose
prisma/schema.prisma	+16 / -0	@@map(...) added to all 8 enums (bug fix)
prisma/migrations/0_init/migration.sql	+408	Baseline migration (new)
prisma/migrations/README.md	+40	Why/how of baselining (new)
prisma/migrations/migration_lock.toml	+3	Prisma migration-history lock file (new)
scripts/generate_baseline_migration.py	+127	pg_dump → migration.sql reformatter (new)
src/prisma.ts	+18	Shared PrismaClient instance (new)
src/demo-queries.ts	+62	Runnable demo of the repository layer (new)
src/repositories/student.repository.ts	+87	Digital Twin, roster, transactional create (new)
src/repositories/risk.repository.ts	+53	Weak-student radar queries (new)
src/repositories/faculty.repository.ts	+43	Course load, attendance upsert (new)
src/repositories/analytics.repository.ts	+59	Placement/section/backlog stats (new)

13 files changed, 960 insertions(+), 5 deletions(-) — commit cae1f29.

9. Known Schema Note (carried over from schema_v2.sql)

Most foreign-key columns (e.g. `course_enrollments.student_id`) are not NOT NULL in the current DDL, so their Prisma fields are modeled as optional to stay byte-for-byte accurate to the real database. This was not introduced in Go 2 — it is flagged here as a reasonable future tightening the team could choose to make at the DB level, rather than silently changing an already-reviewed schema.

10. Go 3 (Week 3) — Coming Next

`prisma/seed.ts` — a synthetic dataset (500+ student profiles and related records) generated and inserted through Prisma Client, wired up to `npx prisma db seed`.

UEI — Unified Education Interface · Prisma ORM layer over schema_v2.sql (14 tables) · Go 2 of 3 weekly submissions